Discovery Executive Summary

Project: dicovery-123 · **Generated:** 24/07/2026, 18:36:40

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 2 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—
2	Performance & Sustainability Analysis	HIGH RISK	—

---

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service/Repository layers (H2/H3), Domain Boundary Violations (H8), Shared DB Coupling (H9), and Dual-API duplication (H10).

EXECUTIVE SUMMARY

Klearcom is a modular monolith (Discovery IVR + Connect TFN) with a thin Laravel 12 API, a React 19 SPA, and a parallel Express `dev-api` that re-implements the same workflows. Controllers are short by LOC but fat by responsibility: Eloquent and KPI math live in HTTP handlers, Modules folders are documentation stubs only, and there are zero repository classes. The dominant risk is change amplification from duplicated business logic across Laravel, Express, and the SPA, plus shared Mongo collections and cross-domain dashboard/legacy reports that erase bounded-context ownership. Frontend has a usable `api/client` and React Query, but legacy class/widget patterns and hard-coded endpoint strings in pages still couple UI to backend shapes.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	62 LOC (Laravel avg); Express `server.js` 224 LOC	MODE
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	25 handler methods with direct model/store access	HIGH
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	35+ Eloquent/store/Mongo access points; 0 repositories	HIGH
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	2 (`LegacyDataMapper` , shared `buildTree` helpers)	MODE
H6	Direct SQL in Controllers	ORM/repository compliance %	>90%	60–90%	<60%	~38% of queries outside controllers	HIGH
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (largest `server.js` 224 LOC)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8 cross-domain access points	HIGH
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	37.5% (3/8 stores shared via Mongo `module`)	HIGH
H10	Dual API Duplication (additional)	Duplicated workflows across Laravel ↔ Express	0	1–3	>3	5+ workflows duplicated	HIGH
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	74 LOC avg	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	6 components with hard-coded paths	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (largest ConnectPage 208 LOC)	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	2 levels; small Zustand UI store	GOOD
F5	Legacy / Inconsistent	Legacy-pattern components	0	1–10	>10	2 (`LegacyMonitorPoller` , `LegacyDashboardWidget`)	MODE

	Component Patterns						

1.4 Actions Required

Hotspot	Action	Rating	Priority
H1 Fat Controllers	Extract reachability/tree/KPI logic from `ConnectController`, `LegacyReportController`, and Express `server.js` into Application Services	MODERATE	HIGH
H2 Missing Service Layer	Create `DiscoveryJobService`, `ConnectMonitorService`, `DashboardKpiService`; stop Eloquent/store use in handlers	HIGH RISK	CRITICAL
H3 Missing Repository Pattern	Add repository interfaces for Eloquent models; bind in `AppServiceProvider`; keep Mongo behind repository APIs	HIGH RISK	CRITICAL
H5 Shared Utility Abuse	Replace `LegacyDataMapper` extract() with DTOs; single `IvrTreeBuilder` domain service	MODERATE	MEDIUM
H6 Direct SQL in Controllers	Enforce repository-only persistence; remove Eloquent from `Http/Controllers` and Mongo from Express routes	HIGH RISK	CRITICAL
H8 Domain Boundary Violations	Populate real `Modules/Discovery` & `Modules/Connect`; move Reporting behind published DTOs/ACL	HIGH RISK	CRITICAL
H9 Shared Database Coupling	Split Mongo collections per BC; document MariaDB ownership; ban cross-module writes	HIGH RISK	CRITICAL
H10 Dual API Duplication	Consolidate on Laravel as single API; proxy or delete Express `dev-api` duplication	HIGH RISK	CRITICAL
F5 Legacy Component Patterns	Convert `LegacyMonitorPoller` to hooks; add Error Boundaries; remove unused legacy widget	MODERATE	MEDIUM

1.5 Expected Outcomes

- Controllers become thin HTTP adapters; Application/Domain Services own workflows and are reusable from jobs/CLI.
- Repository interfaces enable testing without MariaDB/Mongo and isolate schema churn to one layer.
- Real bounded contexts (Discovery, Connect, Reporting) stop silent cross-module coupling and support future extraction.
- Eliminating the dual Laravel/Express stack removes the largest source of divergent business rules.
- Frontend domain API modules plus purged legacy components keep the SPA aligned with a single backend contract.

---

2. Performance & Sustainability Analysis

OVERALL CODEBASE RATING — PERFORMANCE & SUSTAINABILITY

High Risk

Driven by P2 Database, P3 API, P4 Memory, P6 Concurrency, P8 Resource Utilization, P9 Network, and P12 Sustainability.

EXECUTIVE SUMMARY

Klearcom's dual runtime (Laravel + Node `dev-api`) is a small voice-observability platform with clear efficiency debt concentrated in data access, long-lived streaming, and always-on Docker resources. The highest-risk patterns are an N+1 query loop in `LegacyReportController`, unbounded MariaDB list loads, SSE endpoints that re-read full Mongo event histories every 500 ms while holding request workers with `usleep`, and a five-service Compose stack with no CPU/memory limits that ships the frontend via `npm run dev`. Frontend polling (`refetchInterval`, `LegacyMonitorPoller` every 3 s, `MongoStatus` every 15 s) amplifies traffic while nginx lacks gzip. CI installs Composer/npm and pecl MongoDB with no dependency caching. Overall rating is **High Risk**, driven by database, API latency, memory retention, concurrency, network chatter, resource waste, and sustainability posture.

8.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
P1	Algorithm Efficiency	High-complexity algorithm sites	0	1–5	>5	3	MODERATE
P2	Database Performance	Slow-query / N+1 sites	0	1–5	>5	6	HIGH RISK
P3	API Performance	Response-latency hotspots	0	1–5	>5	6	HIGH RISK
P4	Memory Efficiency	High-memory sites	0	1–3	>3	4	HIGH RISK
P5	CPU Efficiency	CPU-intensive operations	0	1–5	>5	3	MODERATE
P6	Concurrency	Blocking / sequential sites	0	1–5	>5	6	HIGH RISK
P7	Caching	Missed caching opportunities	0	1–5	>5	5	MODERATE
P8	Resource Utilization	Over-provisioned / idle resources	0	1–3	>3	4	HIGH RISK
P9	Network Efficiency	Excessive-traffic sites	0	1–5	>5	6	HIGH RISK
P10	Build Efficiency	Build/test pipeline efficiency	efficient	partial	slow / no caching	partial	MODERATE

P11	Logging Efficiency	Excessive-logging sites	0	1–10	>10	1	MODERATE
P12	Sustainability	Resource-optimization posture	optimized	partial	wasteful	wasteful	HIGH RISK
P13	Missing client timeouts (additional)	Fetch/HTTP calls without timeout	0	1–2	>2	1	MODERATE
P14	Uncleared polling intervals (additional)	Intervals without unmount cleanup	0	1	≥2	1	MODERATE

8.5 Actions Required

Hotspot	Action	Rating	Priority
P2 Database Performance	Eliminate N+1 in `LegacyReportController`; paginate list endpoints; add indexes on status/country/checked_at; cursor-limit Mongo event reads	HIGH RISK	CRITICAL
P3 API Performance	Move simulated tests to queue workers; replace SSE Mongo re-poll with pub/sub; stop parallel REST refetch during live runs	HIGH RISK	CRITICAL
P6 Concurrency	Stop holding PHP-FPM/Node with `usleep` /interval polls; introduce worker pool and push-based streams	HIGH RISK	CRITICAL
P4 Memory Efficiency	Cap `getTestEvents` and in-memory store growth; paginate Eloquent collections	HIGH RISK	HIGH
P8 Resource Utilization	Add Compose resource limits; ship static frontend image; disable APP_DEBUG outside local; unpublish DB ports in non-dev	HIGH RISK	HIGH
P9 Network Efficiency	Prefer SSE-only live updates; remove legacy 3 s/10 s pollers; enable nginx gzip/brotli	HIGH RISK	HIGH
P12 Sustainability	Right-size always-on stack and cut wasteful poll/N+1 paths; plan autoscaled workers for AWS	HIGH RISK	HIGH
P1 Algorithm Efficiency	Replace recursive full-scan `buildTree` with parent_id hash grouping (O(n)) shared across Laravel and `dev-api`	MODERATE	MEDIUM
P5 CPU Efficiency	Ping-only health checks; avoid full collection counts on the 15 s status path	MODERATE	MEDIUM
P7 Caching	Cache dashboard KPIs and IVR trees; short-circuit repeated health work	MODERATE	MEDIUM
P10 Build Efficiency	Cache Composer/npm and avoid pecl rebuild every CI run	MODERATE	MEDIUM
P11 Logging Efficiency	Default `APP_DEBUG=false` outside local Compose profile	MODERATE	MEDIUM
P13 Missing client timeouts	Add `AbortSignal.timeout` to `frontend/src/api/client.ts`	MODERATE	MEDIUM
P14 Uncleared polling intervals	Clear `LegacyMonitorPoller` interval on unmount	MODERATE	MEDIUM

8.6 Expected Outcomes

- Batched/indexed MariaDB access and paginated lists remove N+1 and unbounded payload latency as monitors and jobs scale.
- Queue-backed tests plus pub/sub SSE free PHP-FPM/Node workers, raising concurrent Discovery/Connect throughput.
- O(n) tree builds, capped Mongo reads, and server-side KPI/tree caches cut CPU, memory, and repeated query cost.
- gzip, SSE-only live updates, and removal of leaked/legacy pollers reduce chatty traffic and bandwidth.
- Right-sized Compose (static FE, debug off, limits) plus CI dependency caching lower cloud cost, idle energy use, and carbon footprint.